MERN STACK PROJECT · 3 ACTIVITIES

ShowCase: Product Gallery with Image Upload

One project in three activities: Frontend, Backend, and Hosting.

Activity 1 · Frontend → Activity 2 · Backend → Activity 3 · Hosting

M	**E**	**R**	**N**
MongoDB Atlas	Express.js	React + Tailwind	Node.js
Online database	API on Render	Website on Vercel	Runtime

NU MOA
School of Information Technology
BS Information Technology

Student Information

Name

Section / Year

GitHub Repo URL

Render (API) URL

Vercel (Website) URL

Scores

Activity 1 · Frontend	/ 100
Activity 2 · Backend	/ 100
Activity 3 · Hosting	/ 100

3	**42**	**4–5**
Activities	Steps	Lab sessions

ACTIVITY 1 OF 3

Activity 1 – Frontend

Build the website design with React and Tailwind CSS.

A1

Steps 1 – 16

Activity 1 · Frontend | Activity 2 · Backend | Activity 3 · Hosting

Before You Start

- A computer with internet.
- About 1–2 lab sessions.

How to Read Each Step

1 **Step number** – do the steps in order.

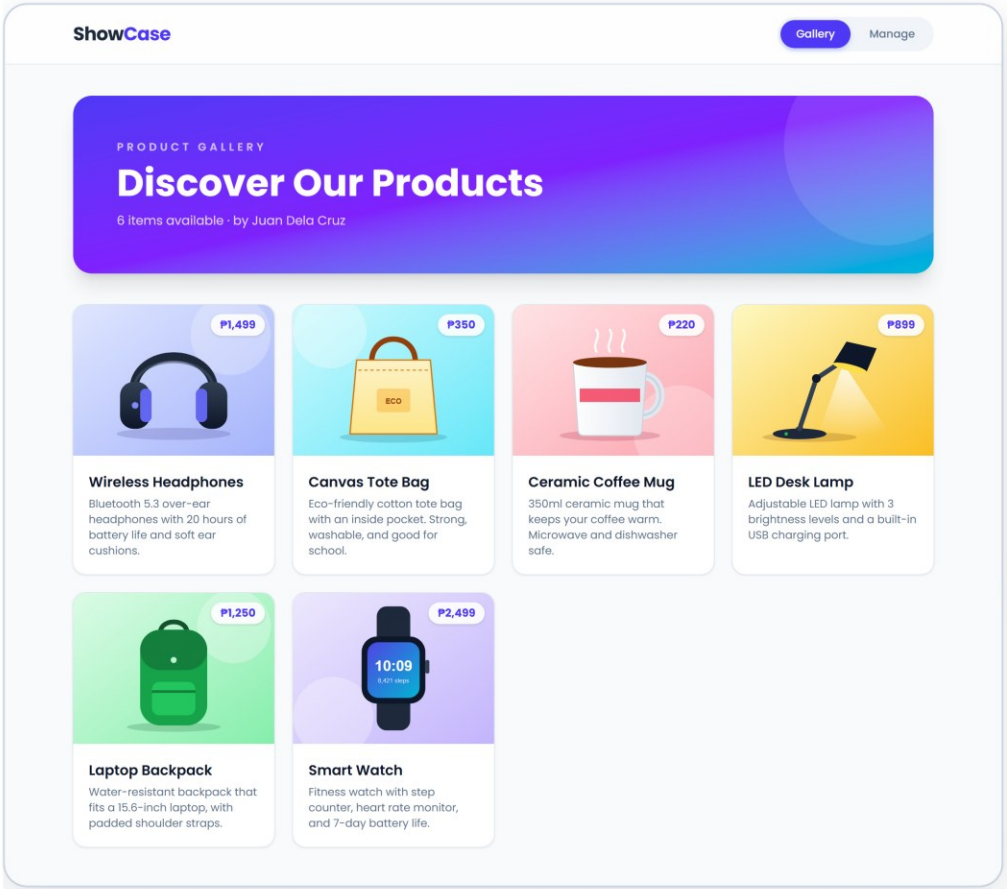
RUN IN **Terminal folder** – where to type the commands.

`client/src/App.jsx` **File** – where to create or edit the file.

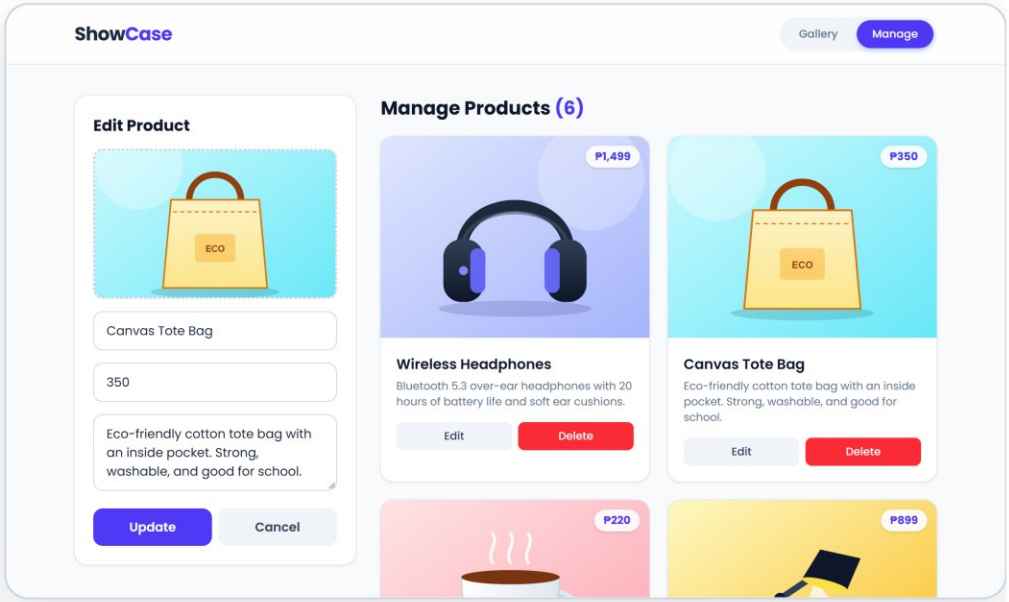
✓ Check **Check** – what you should see if it worked.

1 · Start here **Long code** – read the left box first, then the right box.

What You Will Build



Gallery page – shows every product



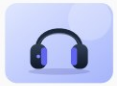
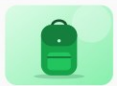
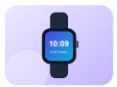
Manage page – add, edit, and delete products

Every Product Needs

Field	Rule
Image	jpg, png, or webp · max 1MB
Name	Required
Price	Number (example: 1499)
Description	1–2 complete sentences

Example Products

Use these as a guide. Make your own products with your own images and complete descriptions.

Image	Name	Price	Description
	**Wireless Headphones**	₱1,499	Bluetooth 5.3 over-ear headphones with 20 hours of battery life and soft ear cushions.
	**Canvas Tote Bag**	₱350	Eco-friendly cotton tote bag with an inside pocket. Strong, washable, and good for school.
	**Ceramic Coffee Mug**	₱220	350ml ceramic mug that keeps your coffee warm. Microwave and dishwasher safe.
	**LED Desk Lamp**	₱899	Adjustable LED lamp with 3 brightness levels and a built-in USB charging port.
	**Laptop Backpack**	₱1,250	Water-resistant backpack that fits a 15.6-inch laptop, with padded shoulder straps.
	**Smart Watch**	₱2,499	Fitness watch with step counter, heart rate monitor, and 7-day battery life.

PART A

Setup

Install the tools and create the project folder.

1 Install the tools

RUN IN `any folder`

1. Install **Node.js** (LTS version) from `nodejs.org`
2. Install **Git** from `git-scm.com`
3. Install **Visual Studio Code** from `code.visualstudio.com`
4. Open VS Code, then **Terminal** → **New Terminal**, and type the commands below.

Terminal · any folder

```
node -v
git --version
```

✓ **Check** Both commands show a version number (example: `v22.11.0`).

2 Create the project folder

RUN IN `any folder (example: Documents)`

1. Type the commands below.
2. VS Code opens a new window. Use this window from now on.

Terminal · any folder (example: Documents)

```
mkdir showcase-gallery
cd showcase-gallery
code .
```

✓ **Check** The VS Code sidebar shows `SHOWCASE-GALLERY` .

PART B

React App

Build the website with React, Vite, and Tailwind CSS.

3 Create the React app

RUN IN `showcase-gallery`

1. Open a terminal: **Terminal → New Terminal**.
2. Type the commands below.
3. If Vite asks extra questions, choose **No**.

Terminal · showcase-gallery

```
npm create vite@latest client -- --template react
cd client
npm install
```

✓ **Check** A `client` folder appears in the sidebar. The terminal is now inside `client`.

4 Install Tailwind CSS

RUN IN `showcase-gallery/client`

Type the command below.

Terminal · showcase-gallery/client

```
npm install tailwindcss @tailwindcss/vite
```

✓ **Check** No red error messages.

5 Add Tailwind to Vite

`client/vite.config.js`

Open this file, delete everything inside, type the code below, and save (**Ctrl + S**).

JS

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

6 Set up the styles

`client/src/index.css`

Open this file, delete everything inside, type the code below, and save (**Ctrl + S**).

CSS

```
@import url("https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&display=swap");
@import "tailwindcss";

@theme {
  --font-sans: "Poppins", sans-serif;
}
```

7 Prepare the folders

1. Delete the file `client/src/App.css`
2. Create the folder `client/src/components`
3. Create the folder `client/src/pages`

✓ **Check** Inside `client/src` you have: `components`, `pages`, `App.jsx`, `index.css`, `main.jsx`.

8 Create the Navbar

`client/src/components/Navbar.jsx`

Create this file, type the code below, and save (**Ctrl + S**).

JSX

```jsx
function Navbar({ view, onChangeView }) {
  const tabClass = (name) =>
    `rounded-full px-5 py-2 text-sm font-medium transition ${
      view === name
        ? "bg-indigo-600 text-white shadow-md shadow-indigo-200"
        : "text-slate-500 hover:bg-white hover:text-slate-900"
    }`;

  return (
    <nav className="sticky top-0 z-20 border-b border-slate-200 bg-white/80 backdrop-blur-md">
      <div className="mx-auto flex max-w-6xl items-center justify-between px-6 py-4">
        <h1 className="text-2xl font-bold tracking-tight">
          Show<span className="text-indigo-600">Case</span>
        </h1>

        <div className="flex gap-1 rounded-full bg-slate-100 p-1">
          <button className={tabClass("gallery")} onClick={() => onChangeView("gallery")}>
            Gallery
          </button>
          <button className={tabClass("manage")} onClick={() => onChangeView("manage")}>
            Manage
          </button>
        </div>
      </div>
    </nav>
  );
}

export default Navbar;
```

9 Create the ProductCard

client/src/components/ProductCard.jsx

Create this file, type the code below, and save (**Ctrl + S**).

JSX

```jsx
function ProductCard({ product, showActions, onEdit, onDelete }) {
  return (
    <article
      className="group overflow-hidden rounded-2xl bg-white shadow-sm ring-1 ring-slate-200
        transition duration-300 hover:-translate-y-1 hover:shadow-xl"
    >
      <div className="relative aspect-4/3 overflow-hidden">
        <img
          src={product.image}
          alt={product.name}
          className="h-full w-full object-cover transition duration-500 group-hover:scale-105"
        />
        <span className="absolute right-3 top-3 rounded-full bg-white/90 px-3 py-1 text-sm font-bold text-indigo-600 shadow">
          ₱{Number(product.price).toLocaleString()}
        </span>
      </div>

      <div className="p-5">
        <h3 className="text-lg font-semibold text-slate-900">{product.name}</h3>
        <p className="mt-1 text-sm text-slate-500">{product.description}</p>

        {showActions && (
          <div className="mt-4 flex gap-2">
            <button
              onClick={() => onEdit(product)}
              className="flex-1 rounded-lg bg-slate-100 py-2 text-sm font-medium text-slate-700 hover:bg-slate-200"
            >
              Edit
            </button>
            <button
              onClick={() => onDelete(product._id)}
              className="flex-1 rounded-lg bg-red-500 py-2 text-sm font-medium text-white hover:bg-red-600"
            >
              Delete
            </button>
          </div>
        )}
      </div>
    </article>
  );
}

export default ProductCard;
```

10 Create the ProductGrid

client/src/components/ProductGrid.jsx

Create this file, type the code below, and save (**Ctrl + S**).

JSX

```jsx
import ProductCard from "./ProductCard";

function ProductGrid({ products, showActions = false, onEdit, onDelete }) {
  if (products.length === 0) {
    return (
      <div className="rounded-2xl border-2 border-dashed border-slate-300 py-20 text-center text-slate-400">
        No products yet.
      </div>
    );
  }

  return (
    <div className="grid grid-cols-[repeat(auto-fill,minmax(230px,1fr))] gap-6">
      {products.map((product) => (
        <ProductCard
          key={product._id}
          product={product}
          showActions={showActions}
          onEdit={onEdit}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default ProductGrid;
```

11 Create the ImageUpload

`client/src/components/ImageUpload.jsx`

Create this file, type the code below, and save (**Ctrl + S**).

JSX

```jsx
const MAX_SIZE = 1024 * 1024;

function ImageUpload({ image, onChange, onError }) {
  const handleFile = (e) => {
    const file = e.target.files[0];
    if (!file) return;
    if (!file.type.startsWith("image/")) return onError("Please choose an image file.");
    if (file.size > MAX_SIZE) return onError("Image is too large. Maximum size is 1MB.");
    const reader = new FileReader();
    reader.onload = () => onChange(reader.result);
    reader.readAsDataURL(file);
    onError("");
  };

  return (
    <label
      className="flex h-48 cursor-pointer items-center justify-center overflow-hidden rounded-xl
        border-2 border-dashed border-slate-300 text-sm text-slate-400 transition
        hover:border-indigo-500 hover:bg-indigo-50 hover:text-indigo-600"
    >
      {image ? (
        <img src={image} alt="Preview" className="h-full w-full object-cover" />
      ) : (
        <span>+ Click to choose an image (max 1MB)</span>
      )}
      <input type="file" accept="image/*" onChange={handleFile} hidden />
    </label>
  );
}

export default ImageUpload;
```

12 Create the ProductForm

`client/src/components/ProductForm.jsx`

Create this file, type the code below, and save (**Ctrl + S**).

1 · Start here

```jsx
import { useState } from "react";
import ImageUpload from "./ImageUpload";

const emptyForm = { name: "", price: "", description: "", image: "" };
const inputClass =
  "w-full rounded-xl border border-slate-300 px-4 py-3 outline-none " +
  "transition focus:border-indigo-500 focus:ring-4 focus:ring-indigo-100";

function ProductForm({ editingProduct, onSubmit, onCancel }) {
  const [form, setForm] = useState(editingProduct || emptyForm);
  const [error, setError] = useState("");
  const [saving, setSaving] = useState(false);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const formElement = e.target;
    const { name, price, description, image } = form;
    if (!name.trim() || price === "" || !image) {
      return setError("Name, price, and image are required.");
    }
    setSaving(true);
    try {
      await onSubmit({
        name: name.trim(), price: Number(price), description, image,
      });
      setForm(emptyForm);
      setError("");
      formElement.reset();
    } catch {
      setError("Could not save the product. Please try again.");
    } finally {
      setSaving(false);
    }
  };

  return (
    <form
      onSubmit={handleSubmit}
      className="space-y-4 rounded-2xl bg-white p-6 shadow-sm ring-1
        ring-slate-200 lg:sticky lg:top-24"
    >
      <h2 className="text-xl font-bold text-slate-900">
        {editingProduct ? "Edit Product" : "Add Product"}
      </h2>
```

2 · Continue here

```jsx
      <ImageUpload
        image={form.image}
        onChange={(image) => setForm((prev) => ({ ...prev, image }))}
        onError={setError}
      />

      <input name="name" placeholder="Product name" className={inputClass}
        value={form.name} onChange={handleChange} />
      <input name="price" type="number" min="0" placeholder="Price (₱)"
        className={inputClass} value={form.price} onChange={handleChange} />
      <textarea name="description" rows="3" placeholder="Short description"
        className={inputClass} value={form.description}
        onChange={handleChange} />

      {error && (
        <p className="rounded-xl bg-red-50 p-3 text-sm text-red-600">{error}</p>
      )}

      <div className="flex gap-2">
        <button
          type="submit"
          disabled={saving}
          className="flex-1 rounded-xl bg-indigo-600 py-3 font-semibold
            text-white transition hover:bg-indigo-700 disabled:opacity-50"
        >
          {saving ? "Saving..." : editingProduct ? "Update" : "Add Product"}
        </button>
        {editingProduct && (
          <button
            type="button"
            onClick={onCancel}
            className="flex-1 rounded-xl bg-slate-100 py-3 font-semibold
              text-slate-700 transition hover:bg-slate-200"
          >
            Cancel
          </button>
        )}
      </div>
    </form>
  );
}

export default ProductForm;
```

13 Create the GalleryPage

client/src/pages/GalleryPage.jsx

Create this file, type the code below, and save. Change `[Your Name]` to your real name.

JSX

```jsx
import ProductGrid from "../components/ProductGrid";

function GalleryPage({ products, loading }) {
  return (
    <main className="mx-auto max-w-6xl px-6 py-10">
      <section
        className="relative mb-10 overflow-hidden rounded-3xl bg-linear-to-br from-indigo-600
          via-violet-600 to-cyan-500 p-10 text-white shadow-xl md:p-14"
      >
        <div className="absolute -right-16 -top-16 h-64 w-64 rounded-full bg-white/10"></div>
        <p className="text-sm font-semibold uppercase tracking-[0.3em] text-white/70">Product Gallery</p>
        <h2 className="mt-3 text-4xl font-bold md:text-5xl">Discover Our Products</h2>
        <p className="mt-3 text-white/80">
          {products.length} items available · by [Your Name]
        </p>
      </section>

      {loading ? (
        <p className="py-20 text-center text-slate-400">Loading products...</p>
      ) : (
        <ProductGrid products={products} />
      )}
    </main>
  );
}

export default GalleryPage;
```

14 Create the ManagePage

`client/src/pages/ManagePage.jsx`

Create this file, type the code below, and save (**Ctrl + S**).

JSX

```jsx
import ProductForm from "../components/ProductForm";
import ProductGrid from "../components/ProductGrid";

function ManagePage({ products, editingProduct, onSave, onCancel, onEdit, onDelete }) {
  return (
    <main className="mx-auto grid max-w-6xl items-start gap-8 px-6 py-10 lg:grid-cols-[360px_1fr]">
      <ProductForm
        key={editingProduct?._id || "new"}
        editingProduct={editingProduct}
        onSubmit={onSave}
        onCancel={onCancel}
      />

      <section>
        <h2 className="mb-5 text-2xl font-bold text-slate-900">
          Manage Products <span className="text-indigo-600">({products.length})</span>
        </h2>
        <ProductGrid products={products} showActions onEdit={onEdit} onDelete={onDelete} />
      </section>
    </main>
  );
}

export default ManagePage;
```

15 Replace App.jsx (Version 1)

client/src/App.jsx

Open this file, delete everything inside, type the code below, and save. Change `[Your Name]` and `[Section]`.

1 · Start here

```
import { useState } from "react";
import Navbar from "./components/Navbar";
import GalleryPage from "./pages/GalleryPage";
import ManagePage from "./pages/ManagePage";

function App() {
  const [products, setProducts] = useState([]);
  const [view, setView] = useState("gallery");
  const [editingProduct, setEditingProduct] = useState(null);

  const saveProduct = (data) => {
    if (editingProduct) {
      setProducts((prev) =>
        prev.map((p) => (p._id === editingProduct._id ? { ...p,
...data } : p))
      );
      setEditingProduct(null);
    } else {
      setProducts((prev) => [{ _id: crypto.randomUUID(), ...data },
...prev]);
    }
  };

  const deleteProduct = (id) => {
    if (!confirm("Delete this product?")) return;
    setProducts((prev) => prev.filter((p) => p._id !== id));
    if (editingProduct?._id === id) setEditingProduct(null);
  };

  const startEdit = (product) => {
    setEditingProduct(product);
    window.scrollTo({ top: 0, behavior: "smooth" });
  };
```

2 · Continue here

```
  return (
    <div className="min-h-screen bg-slate-50 text-slate-800">
      <Navbar view={view} onChangeView={setView} />

      {view === "gallery" ? (
        <GalleryPage products={products} />
      ) : (
        <ManagePage
          products={products}
          editingProduct={editingProduct}
          onSave={saveProduct}
          onCancel={() => setEditingProduct(null)}
          onEdit={startEdit}
          onDelete={deleteProduct}
        />
      )}

      <footer className="py-10 text-center text-sm text-slate-400">
        Made by [Your Name] · [Section]
      </footer>
    </div>
  );
}

export default App;
```

✓ **Check** In the sidebar, `components` has 5 files and `pages` has 2 files.

16 Run and test the website

RUN IN `showcase-gallery/client`

1. Type the command below, then open `http://localhost:5173` in your browser.
2. Click **Manage**. Add 3 products with an image, name, price, and description (see the Example Products page).
3. Click **Edit** on one product, change it, then click **Update**.
4. Click **Delete** on one product.
5. Click **Gallery** to see your products.

Terminal · showcase-gallery/client

```
npm run dev
```

Note
If you refresh the page, the products disappear. This is normal. In Activity 3 they will be saved in the database.

✓ **Check** Add, Edit, and Delete all work. Press **Ctrl + C** in the terminal to stop.

ACTIVITY 1

Activity 1 – What to Submit

Take these screenshots and submit them with the cover page.

Screenshots to Submit

1. Gallery page with your 3 products.
2. Manage page while editing a product.
3. VS Code sidebar showing the `components` and `pages` folders.

Rubric

Criteria	Description	Pts
Setup	Tools installed, project created	10
Components	All components and pages created correctly	25
Add, Edit, Delete	All three work on the page	25
Image Upload	Preview and 1MB limit work	20
Design	Tailwind styles work, name in hero and footer	20
	TOTAL	**100**

ACTIVITY 2 OF 3

Activity 2 – Backend

Create the MongoDB Atlas database and the Express API.

A2

Steps 17 – 27

Activity 1 · Frontend | **Activity 2 · Backend** | Activity 3 · Hosting

Before You Start

- Activity 1 is finished.
- Open the `showcase-gallery` folder in VS Code.

PART C

Database (MongoDB Atlas)

Create the online database where products and images are saved.

C

17 Log in to MongoDB Atlas

1. Go to `cloud.mongodb.com`.
2. Click **Sign Up** (use your Google account) or log in.
3. If a welcome survey appears, answer it or click **Skip**.

✓ **Check** You see the Atlas dashboard.

18 Create a free cluster

1. Click **Create** (or **+ Create cluster**).
2. Select **Free (M0)**.
3. Name: `Cluster0` · Provider: **AWS** · Region: **Singapore**
4. **Uncheck** "Preload sample dataset".
5. Click **Create Deployment**. Wait 1–3 minutes.

✓ **Check** A **Connect to Cluster0** window appears.

19 Create the database user

1. In the **Connect to Cluster0** window, type a username: `showcaseuser`
2. Type a password with **letters and numbers only** (no `@ # % / :`).
3. Copy the username and password into **Notepad**.
4. Click **Create Database User**, then **Close**.

Note
No window? Go to **Security** → **Database Access** → **+ Add New Database User**, enter the username and password, choose **Read and write to any database**, then **Add User**.

✓ **Check** Your password is saved in Notepad.

20 Allow access from anywhere

1. In the left sidebar, click **Security** → **Network Access**.
2. Click **+ ADD IP ADDRESS**.
3. Click **ALLOW ACCESS FROM ANYWHERE** (it fills in `0.0.0.0/0`).
4. Click **Confirm** and wait about 1 minute.

> **Important**
> This is required. Without it, Render cannot connect to your database in Activity 3.

✓ **Check** The list shows `0.0.0.0/0` with status **Active** .

21 Get the connection string

1. In the left sidebar, click **Clusters**. On Cluster0, click **Connect** → **Drivers**.
2. Copy the connection string and paste it in Notepad.
3. Replace `<db_password>` with your password (remove the `< >`).
4. Type `showcase` after `.mongodb.net/` (before the `?`).

Before

```
mongodb+srv://showcaseuser:<db_password>@cluster0.abcde.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0
```

After (example)

```
mongodb+srv://showcaseuser:MyPass123@cluster0.abcde.mongodb.net/showcase?retryWrites=true&w=majority&appName=Cluster0
```

> **Important**
> Keep this string secret. Do not put it in your code. You will paste it in Render in Activity 3.

✓ **Check** Your final connection string is saved in Notepad.

PART D

Express API

Create the backend that saves products to MongoDB Atlas.

22 Create the server project

RUN IN `showcase-gallery`

1. Open a **new** terminal: **Terminal → New Terminal**.
2. Type the commands below one by one.
3. Inside the new `server` folder, create 4 folders: `config`, `models`, `controllers`, `routes`.

Terminal · showcase-gallery

```
mkdir server
cd server
npm init -y
npm install express mongoose cors dotenv
npm pkg set type=module
npm pkg set scripts.start="node server.js"
```

✓ **Check** `server/package.json` contains `"type": "module"` and `"start": "node server.js"`.

23 Connect to the database

server/config/db.js

Create this file, type the code below, and save (**Ctrl + S**).

JS

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Atlas connected!");
  } catch (error) {
    console.error("MongoDB connection error:", error.message);
    process.exit(1);
  }
};

export default connectDB;
```

24 Create the Product model

server/models/Product.js

Create this file, type the code below, and save (**Ctrl + S**).

1 · Start here

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Product name is required"],
      trim: true,
    },
    price: {
      type: Number,
      required: [true, "Price is required"],
      min: [0, "Price cannot be negative"],
    },
    description: {
```

2 · Continue here

```
      type: String,
      default: "",
      trim: true,
    },
    image: {
      type: String,
      required: [true, "Image is required"],
    },
  },
  { timestamps: true }
);

const Product = mongoose.model("Product", productSchema);

export default Product;
```

25 Create the controller

server/controllers/productController.js

Create this file, type the code below, and save (**Ctrl + S**).

1 · Start here

```
import Product from "../models/Product.js";

export const getProducts = async (req, res) => {
  try {
    const products = await Product.find().sort({ createdAt: -1 });
    res.json(products);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const createProduct = async (req, res) => {
  try {
    const { name, price, description, image } = req.body;
    const product = await Product.create({ name, price, description,
image });
    res.status(201).json(product);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};
```

2 · Continue here

```
export const updateProduct = async (req, res) => {
  try {
    const { name, price, description, image } = req.body;
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      { name, price, description, image },
      { new: true, runValidators: true }
    );
    if (!product) return res.status(404).json({ message: "Product
not found" });
    res.json(product);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

export const deleteProduct = async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ message: "Product
not found" });
    res.json({ message: "Product deleted" });
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};
```

26 Create the routes

`server/routes/productRoutes.js`

Create this file, type the code below, and save (**Ctrl + S**).

JS

```
import { Router } from "express";
import {
  getProducts,
  createProduct,
  updateProduct,
  deleteProduct,
} from "../controllers/productController.js";

const router = Router();

router.route("/").get(getProducts).post(createProduct);
router.route("/:id").put(updateProduct).delete(deleteProduct);

export default router;
```

27 Create the main server file

`server/server.js`

Create this file, type the code below, and save (**Ctrl + S**).

JS

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import connectDB from "./config/db.js";
import productRoutes from "./routes/productRoutes.js";

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json({ limit: "5mb" }));

app.get("/", (req, res) => {
  res.json({ message: "ShowCase API is running!" });
});

app.use("/api/products", productRoutes);

connectDB().then(() => {
  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
});
```

Note
The backend will be put online and tested in Activity 3.

✓ **Check** The `server` folder has: `config/db.js`, `models/Product.js`, `controllers/productController.js`, `routes/productRoutes.js`, `server.js`, `package.json`.

ACTIVITY 2

Activity 2 – What to Submit

Take these screenshots and submit them with the cover page.

Screenshots to Submit

1. Atlas **Network Access** showing `0.0.0.0/0` Active.
2. Atlas **Database Access** showing your user.
3. VS Code sidebar showing all files in the `server` folder.

Rubric

Criteria	Description	Pts
Atlas Setup	Cluster, user, network access, connection string	35
Model	Product.js is correct	15
Controller	productController.js is correct	25
Routes & Server	productRoutes.js, server.js, db.js	25
	TOTAL	**100**

ACTIVITY 3 OF 3

Activity 3 – Hosting

Put the backend on Render and the website on Vercel.

A3

Steps 28 – 42

Activity 1 · Frontend | Activity 2 · Backend | Activity 3 · Hosting

Before You Start

- Activities 1 and 2 are finished.
- Your connection string from Step 21 is in Notepad.
- Open the `showcase-gallery` folder in VS Code.

PART E

GitHub

Upload your project to GitHub.

E

28 Create the .gitignore file

`showcase-gallery/.gitignore`

1. Create the file in the **main** `showcase-gallery` folder (not inside client or server).
2. Type the lines below and save.

PLAIN

```
node_modules/
.env
dist/
```

Note
This stops secret and large files from being uploaded to GitHub.

29 Create a GitHub repository

1. Go to `github.com` and sign up or log in.
2. Click **+** (top right) → **New repository**.
3. Name: `showcase-gallery` · choose **Public**.
4. Do **not** add a README. Click **Create repository**.

✓ **Check** GitHub shows an empty repository page with a URL like `https://github.com/your-username/showcase-gallery.git`.

30 Upload your code

RUN IN `showcase-gallery`

1. If your terminal is inside `client` or `server`, type `cd ..` first.
2. Type the commands below. Use **your** name, email, and repository URL.
3. If a login window appears, choose **Sign in with your browser**.

Terminal · showcase-gallery

```
git config --global user.name "Your Name"
git config --global user.email "your-github-email@example.com"
git init
git add .
git commit -m "Frontend and backend"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/showcase-gallery.git
git push -u origin main
```

✓ **Check** Refresh the GitHub page. You see the `client` and `server` folders.

PART F

Render (Backend Hosting)

Put your API online.

31 Create a Web Service

1. Go to `render.com` and click **Sign up with GitHub**.
2. Click **+ New** → **Web Service**.
3. Select your `showcase-gallery` repository → **Connect**.

> **Note**
> Repository not listed? Click **Configure GitHub** and allow access to it.

32 Fill in the settings and deploy

1. Fill in the settings exactly as shown in the table.
2. Under **Environment Variables**, add the variable below.
3. Click **Deploy Web Service**.

Setting	Value
Name	`showcase-api-yourname`
Language	Node
Branch	`main`
Region	Singapore
Root Directory	`server`
Build Command	`npm install`
Start Command	`npm start`
Instance Type	Free

Environment Variable	Value
`MONGO_URI`	Your connection string from Step 21

33 Check that the API is online

1. Open the **Logs** tab and wait until you see `MongoDB Atlas connected!` and **Your service is live**.
2. Copy your Render URL from the top of the page (example: `https://showcase-api-yourname.onrender.com`).
3. Open `YOUR-RENDER-URL/api/products` in your browser.

> **Note**
> On the free plan, the first visit after 15 minutes can take up to 1 minute to load.

✓ **Check** The browser shows `[]`. Write your Render URL on the cover page.

34 Add a test product with Thunder Client

1. In VS Code, open **Extensions**, search **Thunder Client**, and install it.
2. Open Thunder Client → **New Request**. Choose **POST** and type `YOUR-RENDER-URL/api/products`.
3. Open **Body** → **JSON**, type the text below, then click **Send**.
4. Open `YOUR-RENDER-URL/api/products` in your browser again.

JSON

```
{
  "name": "Test Product",
  "price": 100,
  "description": "My first product saved in MongoDB Atlas.",
  "image": ""
}
```

✓ **Check** Thunder Client shows status **201**, and the browser now shows your Test Product.

PART G

Connect the Website to the API

Make the website load and save products in the database.

35 Create the environment file

`client/.env`

1. Create the file inside the `client` folder.
2. Type the line below using **your** Render URL (no `/` at the end).

PLAIN

```
VITE_API_URL=https://showcase-api-yourname.onrender.com
```

36 Create the API file

client/src/api.js

Create this file, type the code below, and save (**Ctrl + S**).

JS

```
const API_URL = import.meta.env.VITE_API_URL;

if (!API_URL) {
  console.error("VITE_API_URL is missing. Check the .env file or Vercel settings.");
}

async function request(path, options = {}) {
  const res = await fetch(`${API_URL}/api/products${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  if (!res.ok) throw new Error(`Request failed: ${res.status}`);
  return res.json();
}

export const getProducts = () => request("");

export const createProduct = (data) =>
  request("", { method: "POST", body: JSON.stringify(data) });

export const updateProduct = (id, data) =>
  request(`/${id}`, { method: "PUT", body: JSON.stringify(data) });

export const deleteProduct = (id) => request(`/${id}`, { method: "DELETE" });
```

37 Replace App.jsx (Version 2)

client/src/App.jsx

Open this file, delete everything inside, type the code below, and save. Change `[Your Name]` and `[Section]`.

1 · Start here

```jsx
import { useEffect, useState } from "react";
import Navbar from "./components/Navbar";
import GalleryPage from "./pages/GalleryPage";
import ManagePage from "./pages/ManagePage";
import { getProducts, createProduct, updateProduct, deleteProduct } from
"./api";

function App() {
  const [products, setProducts] = useState([]);
  const [view, setView] = useState("gallery");
  const [editingProduct, setEditingProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    getProducts()
      .then((data) => setProducts(data))
      .catch(() => setError("Could not load products. Refresh in 1 minute."))
      .finally(() => setLoading(false));
  }, []);

  const saveProduct = async (data) => {
    if (editingProduct) {
      const updated = await updateProduct(editingProduct._id, data);
      setProducts((prev) =>
        prev.map((p) => (p._id === updated._id ? updated : p))
      );
      setEditingProduct(null);
    } else {
      const created = await createProduct(data);
      setProducts((prev) => [created, ...prev]);
    }
  };

  const removeProduct = async (id) => {
    if (!confirm("Delete this product?")) return;
    try {
      await deleteProduct(id);
      setProducts((prev) => prev.filter((p) => p._id !== id));
      if (editingProduct?._id === id) setEditingProduct(null);
    } catch {
      setError("Could not delete the product.");
    }
  };
```

2 · Continue here

```jsx
  const startEdit = (product) => {
    setEditingProduct(product);
    window.scrollTo({ top: 0, behavior: "smooth" });
  };

  return (
    <div className="min-h-screen bg-slate-50 text-slate-800">
      <Navbar view={view} onChangeView={setView} />

      {error && (
        <div className="mx-auto mt-6 max-w-6xl px-6">
          <p className="rounded-xl bg-red-50 p-4 text-red-600">{error}</p>
        </div>
      )}

      {view === "gallery" ? (
        <GalleryPage products={products} loading={loading} />
      ) : (
        <ManagePage
          products={products}
          editingProduct={editingProduct}
          onSave={saveProduct}
          onCancel={() => setEditingProduct(null)}
          onEdit={startEdit}
          onDelete={removeProduct}
        />
      )}

      <footer className="py-10 text-center text-sm text-slate-400">
        Made by [Your Name] · [Section]
      </footer>
    </div>
  );
}

export default App;
```

38 Test with the online database

RUN IN `showcase-gallery/client`

1. Type the command below and open `http://localhost:5173`.
2. Click **Manage** and add 2 products with images.
3. Refresh the page.

Terminal · showcase-gallery/client

```
npm run dev
```

✓ **Check** The Test Product and your new products are still there after refresh. Press **Ctrl + C** to stop.

39 Upload the changes to GitHub

RUN IN `showcase-gallery`

1. Type `cd ..` to go back to `showcase-gallery`.
2. Type the commands below.

Terminal · showcase-gallery

```
git add .
git commit -m "Connect website to the API"
git push
```

✓ **Check** GitHub shows your latest commit.

PART H

Vercel (Website Hosting)

Put your website online.

40 Import the project

1. Go to `vercel.com` and click **Continue with GitHub**.
2. Click **Add New...** → **Project**.
3. Find `showcase-gallery` and click **Import**.

41 Configure and deploy

1. Fill in the settings exactly as shown in the table.
2. Under **Environment Variables**, add `VITE_API_URL` with your Render URL.
3. Click **Deploy** and wait for **Congratulations!**.

Setting	Value
Project Name	`showcase-yourname`
Root Directory	Click **Edit** → choose `client` → **Continue**
Framework Preset	Vite
`VITE_API_URL`	Your Render URL (no `/` at the end)

✓ **Check** Vercel shows your website URL (example: `showcase-yourname.vercel.app`). Write it on the cover page.

42 Test the live website

1. Open your Vercel URL on your computer.
2. Add products until you have at least **5 products with images**.
3. Open the same URL on your **phone**.
4. Try Edit and Delete on the live website.

✓ **Check** Your products appear on both your computer and your phone, even after refresh. **You are done!**

ACTIVITY 3

Activity 3 – What to Submit

Take these screenshots and submit them with the cover page.

Screenshots to Submit

1. GitHub repository page.
2. Render page showing **Live**.
3. Thunder Client showing status **201**.
4. Live Vercel website with 5+ products.
5. Photo of the website on your phone.

Rubric

Criteria	Description	Pts
GitHub	Code uploaded, no .env or node_modules	15
Render	API is live and connected to Atlas	25
Connection	Website loads and saves data from Atlas	25
Vercel	Website is live	25
Any Device	Works on a phone	10
	TOTAL	**100**

HELP

Common Problems

Find your problem and do the fix.

Problem	Fix
Website has no design	Check Step 5 and Step 6, then stop and run `npm run dev` again.
“Image is too large”	Use an image under 1MB.
Render logs: `bad auth`	Wrong password in the connection string. Fix `MONGO_URI` in Render → Environment.
Render logs: `Could not connect to any servers`	Do Step 20 again (Network Access `0.0.0.0/0`).
Render: `Cannot find module ... server.js`	Render → Settings → Root Directory must be `server`.
Vercel build failed	Vercel → Settings → Root Directory must be `client`.
Website says “Could not load products”	Open your Render URL first and wait 1 minute, then refresh the website.
Live website does not show products	Vercel → Settings → Environment Variables: check `VITE_API_URL`, then **Deployments** → ⋯ → **Redeploy**.
`git push` rejected	Type `git pull origin main --rebase`, then `git push`.